

PHENIKAA UNIVERSITY
PHENIKAA SCHOOL OF COMPUTING



SUBJECT: SOFTWARE ARCHITECTURE
PROJECT FINAL REPORT: MOVIE TICKET BOOKING SYSTEM

Instructor : Vu Quang Dung
Student Name: Bui Minh Duc
Student ID: 23010513
Email: 23010513@st.phenikaa-uni.edu.vn

Course class: CSE703110-1-2-25(N01)

Ha Noi, March 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Goal:	4
1.2 Final Achievement:.....	4
2. ARCHITECTURAL MODELS	4
2.1 1. Overall Architecture	4
2.2 High-Level Service Topology	5
2.3 Detailed Service Models.....	5
2.3.1 3.1 API Gateway.....	5
2.3.2 3.2 Movie Service.....	6
2.3.3 3.3 Booking Service	7
2.3.4 Payment Service	7
2.3.5 User Service.....	8
2.3.6 Notification Service.....	8
2.3.7 Frontend Architecture.....	8
2.4 Data Architecture (ER Model Overview).....	9
2.5 Inter-Service Communication	9
2.5.1 Synchronous (HTTP).....	9
2.5.2 Asynchronous (RabbitMQ)	10
2.6 Architectural Characteristics Summary.....	10
3. COMPLETED TASK DETAILS.....	11
3.1 1. System Architecture (C4 Model).....	11
3.2 Technology Stack Specification	15
3.2.1 Backend & Data.....	15
3.2.2 Frontend.....	15
3.2.3 Infrastructure & DevOps	15
3.3 Port Mapping Management	15
3.4 Project Documentation Audit	16

3.4.1	Available Documentation	16
3.4.2	Missing / Incomplete Documentation.....	16

1. EXECUTIVE SUMMARY

1.1 Goal:

The primary objective of this project was to refactor a legacy Layered Monolith movie ticket booking application into a distributed Microservices Architecture.

Architectural Pattern: The system adopts a Microservices style utilizing Database-per-Service isolation and a Hybrid Communication Pattern (Synchronous REST APIs for reads and Asynchronous AMQP via RabbitMQ for writes).

1.2 Final Achievement:

The team successfully decomposed the application into independent containers (Gateway, Movie, Booking, Payment, Notification). The final system is orchestrated via Docker Compose, demonstrating high scalability, fault tolerance, and a clear separation of concerns, successfully addressing the limitations of the previous monolithic version.

2. ARCHITECTURAL MODELS

2.1 1. Overall Architecture

The system is a microservices-based movie ticket booking application composed of:

- 5 backend services
- 1 API Gateway
- 1 Message Broker (RabbitMQ)
- 4 isolated SQLite databases
- 1 Vanilla JavaScript Single Page Application (SPA) frontend

The system follows three primary architectural patterns:

Pattern	Implementation
Microservices Database-per-Service	Five Flask services and one pure Python consumer, independently deployable. Four services own their own SQLite database.
API Gateway Pattern	Single entry point (Gateway running on port 5005) enforcing security and routing
Event-Driven Architecture	RabbitMQ used for asynchronous notifications via durable queues

2.2 High-Level Service Topology

System Components:

- **Frontend**
 - Browser-based SPA
 - Runs on port 5173
 - Communicates only with the API Gateway
- **API Gateway (Port 5005)**
 - Single externally accessible backend entry point
 - Handles authentication, authorization, routing
- **Backend Services**

Service	Port	Responsibility
User Service	5004	Authentication, registration, token validation
Movie Service	5001	Movies, rooms, showtimes management
Booking Service	5002	Seat management and booking orchestration
Payment Service	5003	Payment processing and card storage
Notification Service	No HTTP port	Event consumer for notifications

- **Databases**

Service	Database
User Service	users.db
Movie Service	movies.db
Booking Service	booking.db
Payment Service	payments.db

- **Message Broker**
 - RabbitMQ (Port 5672)
 - Three durable queues
 - Used for asynchronous communication

2.3 Detailed Service Models

2.3.1 3.1 API Gateway

The API Gateway is the only externally reachable backend component.

- **Responsibilities**

- Validate static API key (peko-key)
- Validate Bearer token via User Service
- Inject user headers (X-User-Email, X-User-Role)
- Forward requests to appropriate service
- **Forwarding Methods**

Method	Authentication	Role Check	Purpose
public_forward()	No	No	Public routes (movies, rooms, seats, user profile, auth)
validate_and_forward()	Yes	Optional (admin)	Protected routes

- **Security Layers**
 1. Static API key validation
 2. Bearer token verification
 3. Role-based access control

The gateway contains no business logic.

2.3.2 3.2 Movie Service

Architecture:

Presentation Layer → Business Logic Layer → Persistence Layer

- **Responsibilities**
 - Manage movies, rooms, and showtimes
 - Validate showtime duration
 - Cascade delete movie → showtimes → bookings
 - Publish refund and schedule change events
- **Database: movies.db**

Tables:

- movies

- rooms
- showtimes

2.3.3 3.3 Booking Service

Architecture:

Presentation Layer → Business Logic Layer → Persistence Layer

- **Responsibilities**
 - Multi-seat grouped booking
 - Tier-based pricing calculation
 - Atomic seat locking
 - Event publishing (OrderPlacedEvent)
- **Key Design Decisions**
 - Grouped booking: multiple seats under one booking ID
 - Atomic seat locking using conditional UPDATE
 - Seat tiers:
 - STANDARD (no surcharge)
 - VIP (+20,000 VND)
 - COUPLE (+50,000 VND)
 - Lazy seat initialization
- **Database: booking.db**

Tables:

 - bookings
 - seats

2.3.4 Payment Service

Flat Flask architecture (no separate layers).

- **Responsibilities**
 1. Fetch booking details
 2. Update booking status to confirmed
 3. Publish INVOICE_PRINT event
- **Database: payments.db**

Table:

 - cards (stored payment methods)

2.3.5 User Service

Flat Flask architecture.

- **Responsibilities**
 - User registration
 - Password hashing
 - Login and token issuance
 - Token verification (used by Gateway)

- **Database: users.db**

Tables:

- users
- tokens

Tokens are UUID-based and do not expire automatically.

2.3.6 Notification Service

Consumer-only process (no HTTP endpoints).

- **Responsibilities**
 - Connect to RabbitMQ
 - Consume three durable queues
 - Dispatch based on event type
 - Simulate email/print via console output
- **Queues**

Queue	Event Type	Producer
ticket_events	OrderPlacedEvent	Booking Service
invoice_events	INVOICE_PRINT	Payment Service
notification_events	REFUND_NOTIFICATION, SHOWTIME_CHANGED	Movie Service

Notification failures do not affect booking or payment transactions.

2.3.7 Frontend Architecture

Vanilla JavaScript SPA built with Vite.

- **Key Components**
 - main.js
- Client-side routing

- Role-based admin route protection
apiClient.js
- Centralized HTTP client
- Attaches API key and token headers
- Auto logout on 401 response

2.4 Data Architecture (ER Model Overview)

- **Core Entities**
- **User Domain**
 - USERS
 - TOKENS
- **Movie Domain**
 - MOVIES
 - ROOMS
 - SHOWTIMES
- **Booking Domain**
 - BOOKINGS
 - SEATS
- **Payment Domain**
 - CARDS

Cross-database relationships are logical only. They are enforced through HTTP calls at the application layer rather than database foreign key constraints.

2.5 Inter-Service Communication

2.5.1 Synchronous (HTTP)

From	To	Purpose
Gateway	User Service	Token verification
Booking Service	Movie Service	Retrieve movie/showtime details
Payment Service	Booking Service	Booking lookup and status update
Movie Service	Booking Service	Cancel bookings on delete

2.5.2 Asynchronous (RabbitMQ)

Producer	Event	Consumer
Booking Service	OrderPlacedEvent	Notification Service
Payment Service	INVOICE_PRINT	Notification Service
Movie Service	REFUND_NOTIFICATION	Notification Service
Movie Service	SHOWTIME_CHANGED	Notification Service

All queues are durable and messages are persistent.

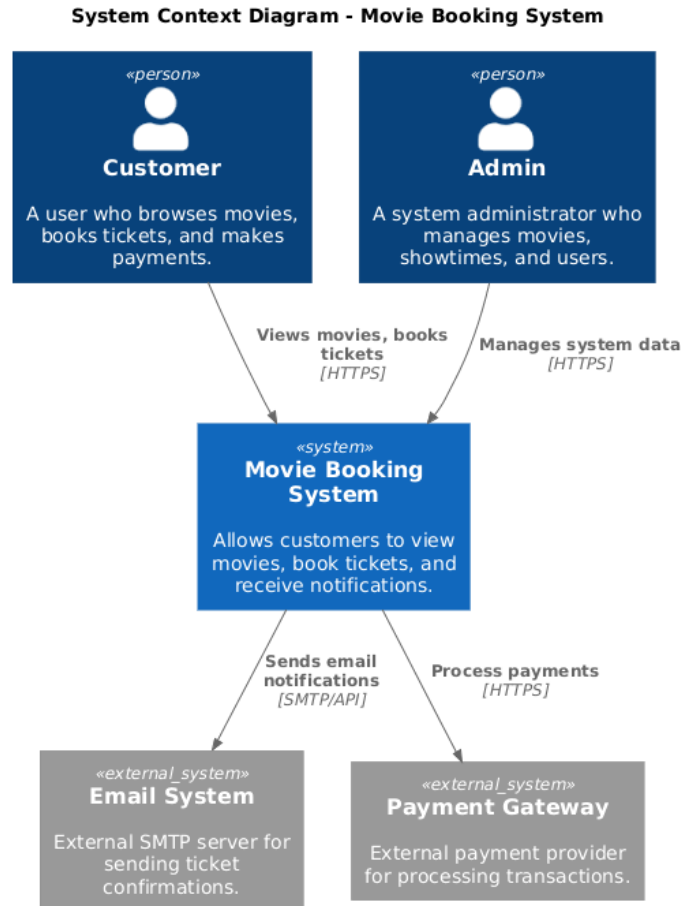
2.6 Architectural Characteristics Summary

Attribute	Implementation
Scalability	Independent service deployment
Isolation	Database-per-service
Consistency	Atomic seat locking
Security	API key + token + role validation
Resilience	Non-blocking async messaging
Deployability	Docker Compose orchestration
Maintainability	Layered design in core services (Movie, Booking); flat architecture in simpler services
Portability	SQLite-based storage

3. COMPLETED TASK DETAILS

3.1 1. System Architecture (C4 Model)

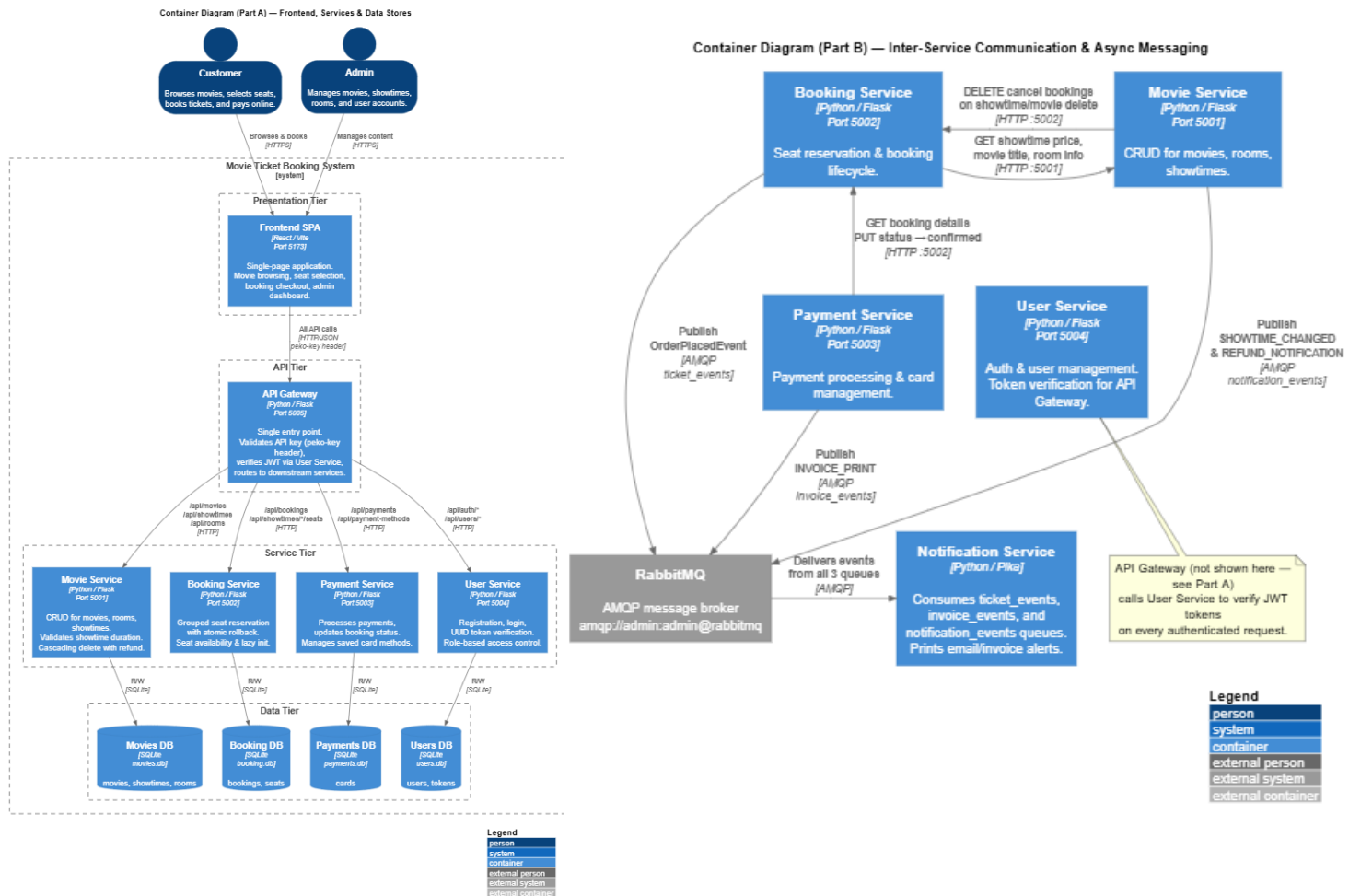
Level 1: System Context Diagram



This is the highest-level view, focusing on how the system interacts with the external environment.

- Core System: Movie Ticket Booking System
- Actors:
 - Customer: Browses movies and showtimes, selects seats, and purchases tickets.
 - Cinema Manager (Administrator): Manages the movie catalog, configures showtimes, and monitors revenue.
- External Systems:
 - Payment Gateway: Processes card transactions (simulated VNPAY/Momo).
 - Email System (SMTP): Sends ticket and invoice notifications to customers via RabbitMQ.
- Objective: Answers the question “*Who does this system serve, and what does it interact with?*”

Level 2: Container Diagram

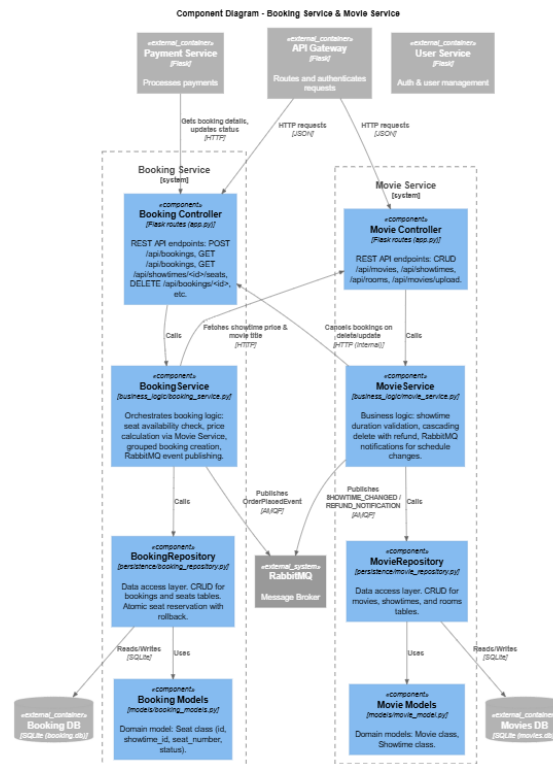


This level decomposes the system into independently deployable units (containers) and clearly illustrates the Microservices architecture.

- Web Application (Frontend): A Single Page Application (Vanilla JS / Vite) running in the user's browser.
- API Gateway: The single entry point that routes requests to downstream services and handles basic security.
- Core Microservices:
 - Movie Service: Manages movies and showtimes.
 - Booking Service: Handles ticket booking and seat state management.
 - Payment Service: Manages payment methods and transactions.
 - User Service: Manages user accounts and authorization.
 - Notification Service: A worker service responsible for sending notifications (consumer).

- Message Broker (***RabbitMQ***): Provides asynchronous communication between services.
- Databases: Each service owns its own SQLite database file (users.db, movies.db, booking.db, etc.).
- Objective: Answers the question “*What are the major building blocks of the system, and how do they communicate (HTTP/AMQP)?*”

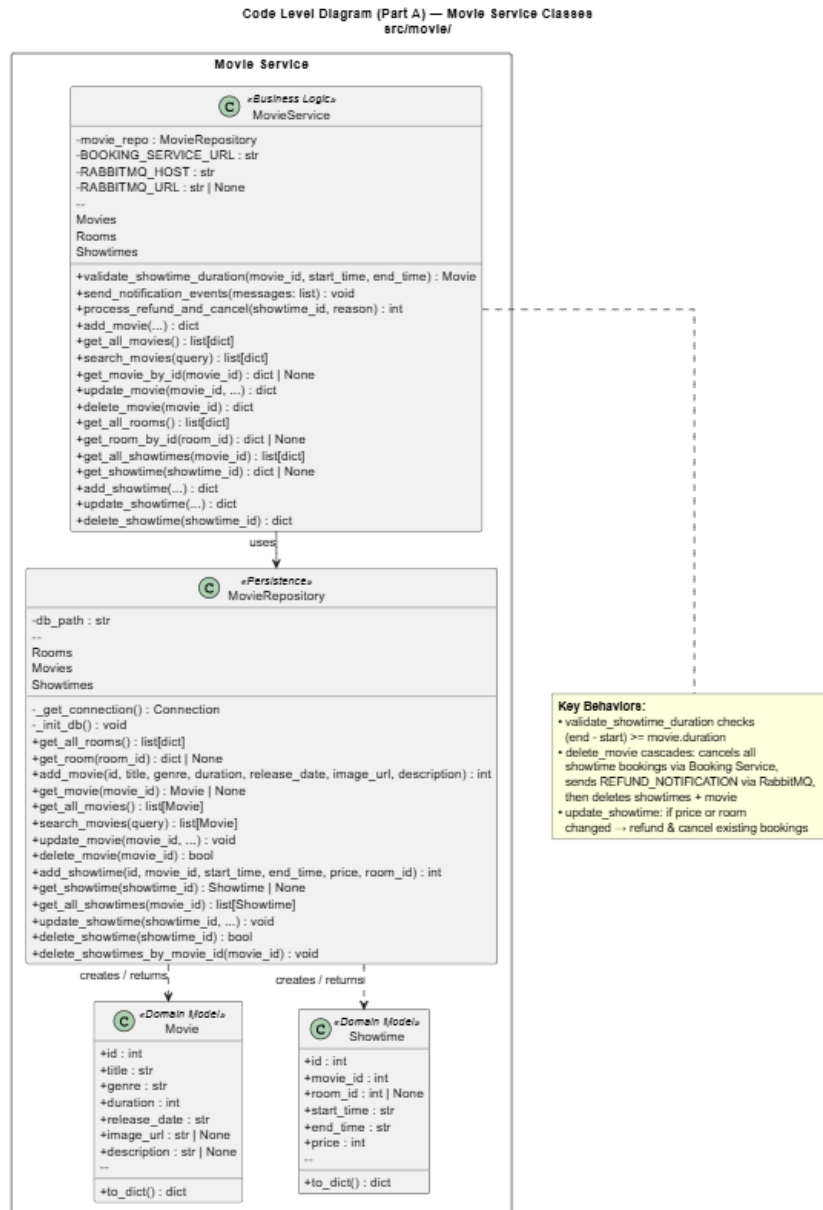
Level 3: Component Diagram



This level dives inside a single container (e.g., the Movie Service) and describes its internal logical components.

- API Controller: Receives requests from the Gateway and validates input parameters.
- Business Service: Implements business logic (e.g., validating whether a showtime duration matches the movie length).
- Persistence Repository: Executes SQL queries against the database.
- Message Publisher: Publishes events to RabbitMQ when data changes occur.
- External Client (Adapter): Invokes APIs of other microservices.
- Objective: Answers the question “*How is a service internally structured into logical components?*”

Level 4: Code Diagram



This is the most detailed level, describing class structures, interfaces, and specific functions.

- **Directory Structure:** Maps components to code (e.g., `persistence/movie_repository.py` implements the `MovieRepository` class).
- **Key Functions:**
 - **`book_ticket(showtime_id, seat_numbers)`:** Handles ticket booking logic in the Booking Service.
 - **`is_seat_available()`:** Checks seat availability in the Booking Repository.
 - **`send_notification_event()`:** Publishes messages to RabbitMQ.
- **Operational Mechanisms:**

- Uses the Repository Pattern to abstract data access.
- Uses the Service Pattern to centralize business logic.
- Uses atomic SQL updates to handle data concurrency.
- Objective: Answers the question “*How is this logic implemented in code by developers?*”

3.2 Technology Stack Specification

The system uses modern technologies with a strong focus on containerization and event-driven communication.

3.2.1 Backend & Data

- Language: Python 3.9
- Framework: Flask (with Flask-CORS)
- Database: SQLite using the database-per-service model
(4 independent databases: ***users.db***, ***movies.db***, ***booking.db***, ***payments.db***)
- Message Broker: RabbitMQ 3 (managed via the pika library)
- Security: ***werkzeug.security*** for password hashing and a custom token system equivalent to JWT

3.2.2 Frontend

- Runtime & Build Tool: Node.js 22 and Vite
- Language: Vanilla JavaScript (ES6+ Modules), avoiding intermediate frameworks for performance optimization
- UI Framework: Bootstrap 5.3.0
- Routing: Custom SPA router based on the History API.

3.2.3 Infrastructure & DevOps

- Containerization: Docker and Docker Compose
- Scale: The system runs 8 containers concurrently
(Frontend, Gateway, 5 Backend Services, RabbitMQ, DB Seeder)
- Design Tool: Draw.io

3.3 Port Mapping Management

To avoid conflicts and ensure smooth inter-service communication, the system assigns ports as follows:

Component	Port	Notes
Frontend	5173	Vite App
API Gateway	5000	Single entry point
Movie Service	5001	Movie & showtime management

Component	Port	Notes
Booking Service	5002	Booking & seat management
Payment Service	5003	Payment processing
User Service	5004	User management
RabbitMQ	5672 / 15672	AMQP / Management UI

3.4 Project Documentation Audit

Based on the current file structure, the documentation status is evaluated as follows:

3.4.1 Available Documentation

- **README.md:** Installation guide and architectural overview
- **checklist.md:** Detailed development roadmap across five phases (Lab 2–8)
- **changelog.md:** Detailed development history from December 2025 to present
- **Frontend_Analysis.md:** In-depth analysis of frontend component structure
- **Wiki Overview:** Comprehensive technical documentation covering databases, security, and deployment

3.4.2 Missing / Incomplete Documentation

- **Executive Summary:** No dedicated document yet
- **System Use Case Specifications:** Detailed use case documents are missing (only diagrams exist)
- **Final Project Report:** Currently being compiled
- **ATAM Analysis:** Planned but not yet conducted (architecture evaluation)
- **Deployment Diagram:** Needs to be updated to reflect the actual Docker deployment.